DBDOME HTTP API - Functionality Reference

Version 1.0.0 · Base URL http://<host>:8080 · 26 endpoints

The DBDOME REST API exposes the data behind the product's dashboards as JSON. Versioned endpoints live under the stable /api/v1 prefix; the unversioned /api/... paths remain for backward compatibility. Every endpoint below is a GET returning application/json.

Conventions

Versioning: Stable endpoints are served under /api/v1. The same handlers are also reachable at the legacy /api/... paths.

Time window (from / to): Most endpoints accept optional from and to query parameters. Each may be Grafana-style epoch milliseconds (e.g. 1784020822440) or an ISO / 'YYYY-MM-DD HH:MM' local datetime. Aggregation endpoints that had a dashboard time filter default to the last 7 days when no window is given.

Paging: List endpoints accept limit (default 500, max 5000).

Response shape: List endpoints return {"<key>": [...], "count": N}; stat endpoints return {"summary": { ... }}; errors return {"error": "..."} with HTTP 500 (or {"detail": "..."} with 400 for bad input).

OpenAPI: Machine-readable spec at /openapi.json (offline-safe). Swagger UI at /docs and ReDoc at /redoc (these load their UI assets from a CDN, so they render only where the browser has internet; the spec itself is always available).

Alerts

GET `/api/v1/alerts/summary` (legacy /api/alerts/summary)

Open Alerts header counts: open total and per-severity open counts (SEC-* root causes) plus the number of monitored servers.

Parameter	Description
from, to	time window (optional)

```
{"summary": {"total": 157, "open": 150, "critical": 5, "high": 38, "medium": 23, "low": 84, "servers": 23}}
```

GET `/api/v1/alerts/open` (legacy /api/alerts/open)

Open Alerts feed - one row per server+root_cause (latest), with server name, root-cause metadata, and mail-sent / blocked timestamps.

Parameter	Description
from, to	time window
risk_level	critical|high|medium|low
server	exact match
root_cause_id	exact match
status	open|resolved
limit	default 500

```
{"alerts": [{"row_id": 9004142, "incident_id": 974, "server": "192.168.1.229,1433", "servername": "mssql2012-1433",
"risk_level": "low", "status": "resolved", "area": "Auditing", "issue": "...", "root_cause_id":
"SEC-SQL-AUD-021-RC01", "root_cause_name": "...", "recipients": null, "occured_at": "2026-07-16 10:15:31",
"sent_at": null, "blocked_at": null}], "count": 1}
```

GET `/api/v1/alerts/incidents` (legacy /api/alerts/incidents)

Alert incidents (open / resolved lifecycle) with resolution metadata and age.

Parameter	Description
status	open|resolved
risk_level	severity
server	exact match
limit	default 500

```
{"incidents": [{"incident_id": 974, "server": "...", "root_cause_id": "...", "risk_level": "high", "status":
"open", "opened_at": "...", "occurrences": 3, "age_hours": 12.5}], "open": 150, "resolved": 7}
```

GET `/api/v1/alerts/log/{alert_id}` (legacy /api/alerts/log/{alert_id})

Active detection findings for one alert (monitoring.get_alert_log_resultset_byid); each finding's result json is expanded.

Parameter	Description
alert_id	path: the alert_log row / alert id
from, to	optional entry_date window


```
{"alert_id": 9004121, "findings": [{"result": {"server": "192.168.1.229:5432", "matched": true, "description":
"Archive command failing...", "row_count": 4, "risk_level": "low", "sample_rows": [ ... ]}, "entry_date":
"2026-07-13 08:16:02"}], "count": 1}
```


GET `/api/v1/alerts/ransomware` (legacy `/api/alerts/ransomware`)

Ransomware Guard feed (SEC-SQL-AUD-031-*): server, risk, login, session id, root-cause description and truncated sql_text.

Parameter	Description
from, to	window (default last 30 days)
server	exact match
risk_level	severity
root_cause_id	defaults to SEC-SQL-AUD-031-% family
limit	default 500


```
{"alerts": [{"time": "2026-07-16 09:12:03", "server": "...", "root_cause_id": "SEC-SQL-AUD-031-RC01", "risk_level":
"critical", "login_name": "app_user", "session_id": "84", "description": "A user session is overwriting data with
encrypted/binary values...", "sql_text": "UPDATE ...", "metric_result_row_id": 8123}], "count": 0}
```


Detection catalogs

GET `/api/v1/policy-enforcement-and-protection` (legacy `/api/policy-enforcement-and-protection`)

Policy Enforcement & Protection catalog: distinct SEC-domain root causes (id, name, description). Customer-facing names/descriptions only, not the detection logic.

Parameter	Description
root_cause_id	optional exact match
vendor_name	default sqlserver


```
{"root_causes": [{"root_cause_id": "SEC-SQL-ACC-010-RC07", "root_cause_name": "After-hours transaction activity",
"root_cause_desc": "Transactions running outside business hours..."}], "count": 515}
```


GET `/api/v1/database-restore` (legacy `/api/database-restore`)

Database restore catalog: SEC-domain root causes in the AZ / CFG / ENC / VS areas (area, issue, id, name, description).

Parameter	Description
area_code	CSV; default AZ,CFG,ENC,VS
root_cause_id	optional exact match


```
{"root_causes": [{"area_name": "Authorization", "issue_name": "Excessive Administrative Privileges",
"root_cause_id": "SEC-SQL-AZ-001-RC01", "root_cause_name": "...", "root_cause_desc": "..."}], "count": 280}
```


Data protection

GET `/api/v1/data-protection/unmasked` (legacy `/api/data-protection/unmasked`)

Data Protection - unmasked sensitive data view (SEC-SQL-PRI-001-RC10): masking support level, encryption/master key counts and masked-column counts per server.

Parameter	Description
server	optional exact match
limit	default 500


```
{"unmasked": [{"server": "181.214.214.254", "edition": "Express Edition (64-bit)", "masking_support_level":
"LIMITED_FEATURES", "masked_column_count": null, "product_version": "16.0.1000.6", "entry_date": "2026-06-22
06:07:26"}], "count": 5}
```


IPS report

GET `/api/v1/ips/summary` (legacy `/api/ips/summary`)

IPS header stats: total / critical / high / medium intrusion events, blocked (FortiAnalyzer IPS) and monitored (mailed).

Parameter	Description
from, to	window (default 7 days)

```
{"summary": {"total": 40128, "critical": 302, "high": 2954, "medium": 3939, "blocked": 0, "monitored": 321}}
```

GET `/api/v1/ips/by-severity` (legacy `/api/ips/by-severity`)

Intrusions grouped by severity.

Parameter	Description
from, to	optional window

```
{"by_severity": [{"severity": "critical", "count": 302}], "count": 4}
```

GET `/api/v1/ips/by-type` (legacy `/api/ips/by-type`)

Top 20 intrusion types (root-cause issue name).

Parameter	Description
from, to	window (default 7 days)

```
{"by_type": [{"type": "SQL Injection attempt", "count": 812}], "count": 20}
```

GET `/api/v1/ips/timeline` (legacy `/api/ips/timeline`)

Intrusion events on an hourly timeline, by severity.

Parameter	Description
from, to	window (default 7 days)

```
{"timeline": [{"time": "2026-07-16 09:00:00", "count": 14, "severity": "high"}], "count": 511}
```

GET `/api/v1/ips/monitored` (legacy `/api/ips/monitored`)

Monitored intrusions by attack / type.

Parameter	Description
from, to	window (default 7 days)

```
{"monitored": [{"attack": "...", "type": "...", "count": 42}], "count": 729}
```

GET `/api/v1/ips/blocked` (legacy `/api/ips/blocked`)

Blocked intrusions by attack / type / severity (top 20).

Parameter	Description
from, to	window (default 7 days)

```
{"blocked": [{"attack": "...", "type": "...", "severity": "critical", "count": 9}], "count": 13}
```

GET `/api/v1/ips/top-victims` (legacy `/api/ips/top-victims`)

Top victim servers by attack / type.

Parameter	Description
from, to	optional window

```
{"top_victims": [{"server": "...", "attack": "...", "type": "...", "count": 30}], "count": 75}
```

GET `/api/v1/ips/top-sources` (legacy `/api/ips/top-sources`)

Top attack sources as a percentage of total, with the matched metric metadata.

Parameter	Description
from, to	window (default 7 days)

```
{"top_sources": [{"pct_of_total": 3.2, "metric_metadata": "{...}", "risk_level": "high"}], "count": 11505}
```

Retention

GET `/api/v1/retention/overview` (legacy `/api/retention/overview`)

Retention overview per category / item: data size, byte size, row count, free space and configured dump-retention months.


```
{"retention": [{"category": "...", "item": "SEC-SQL-...", "root_cause_name": "...", "data_size": "12 MB",
"data_bytes": 12582912, "row_count": 40311, "space_free": "...", "retention_months": 12}], "count": 1669}
```


GET `/api/v1/retention/dump-metrics` (legacy `/api/retention/dump-metrics`)

Configured per-metric dump retention (config.dump_metrics).


```
{"dump_metrics": [{"row_id": 1, "metric_name": "SEC-SQL-...", "retention_months": 12, "entry_date": "..."}],
"count": 1636}
```


GET `/api/v1/retention/dumps` (legacy `/api/retention/dumps`)

Archived dumps catalog (config.dumps).


```
{"dumps": [{"row_id": 1, "metric_name": "...", "month": 6, "year": 2026, "row_count": 1200, "dump_location": "...",
"entry_date": "..."}], "count": 0}
```


GET `/api/v1/retention/dump-location` (legacy `/api/retention/dump-location`)

Current dump / archive location.


```
{"dump_location": [{"dump_location": "D:/dbdome/dumps"}], "count": 1}
```


Blocker activity

GET `/api/v1/blocker/summary` (legacy `/api/blocker/summary`)

Blocker action counts (killed / dry-run / skipped / errors) and the current dry-run mode.

Parameter	Description
from, to	window (default 7 days)


```
{"summary": {"killed": 0, "dry_run": 0, "skipped": 0, "errors": 0, "dry_run_mode": "true"}}
```


GET `/api/v1/blocker/log` (legacy `/api/blocker/log`)

Blocker activity log (alerts.blocker_log): server, vendor, root cause, session, action, detail.

Parameter	Description
from, to	window (default 7 days)
limit	default 500


```
{"blocker_log": [{"entry_date": "...", "server": "...", "db_vendor": "sqlserver", "root_cause_id": "...",
"session_id": "84", "action": "killed", "detail": "..."}], "count": 0}
```


GET `/api/v1/blocker/blocks` (legacy `/api/blocker/blocks`)

Executed blocks (alerts.blocks): server, metric, subject, body.

Parameter	Description
limit	default 200


```
{"blocks": [{"entry_date": "...", "server": "...", "metric_name": "...", "subject": "...", "body": "..."}],
"count": 0}
```


Same login from multiple hosts

GET `/api/v1/same-login-multihost/findings` (legacy `/api/same-login-multihost/findings`)

Detailed findings for the same-login-from-multiple-hosts detection (SEC-SQL-ACC-010-RC10): login, host count, hosts, session count, queries.

Parameter	Description
from, to	window (default 7 days)


```
{"findings": [{"server": "...", "host_count": 3, "hosts": "...", "login_name": "app_user", "session_count": 7,
"entry_date": "...", "queries": "..."}], "count": 226}
```

GET `/api/v1/same-login-multihost/root-causes` (legacy `/api/same-login-multihost/root-causes`)

Root cause(s) behind the same-login-multi-host detection.

```
{"root_causes": [{"root_cause_id": "SEC-SQL-ACC-010-RC10", "step_name": "...", "root_cause_desc": "...",
"risk_level": "high"}], "count": 1}
```

Performance

GET `/api/v1/performance/stored-proc-slow` (legacy `/api/performance/stored-proc-slow`)

Stored procedures running slower than their historical average (SEC-SQL-QE-001-RC01): procedure, database, login, avg vs actual duration.

Parameter	Description
from, to	window (default 7 days)
server	optional
limit	default 500

```
{"slow_procedures": [{"server": "...", "procedure_name": "usp_daily_bill", "database_name": "billing",
"login_name": "svc_app", "avg_duration_ms": 120, "actual_duration_ms": 5400, "entry_date": "..."}], "count": 0}
```

Complete endpoint reference (240 endpoints)

Auto-extracted from the live service (http_server.py), grouped by feature area.

Alerts, Thresholds & Webhooks (23)

GET `/api/alert-rules`

Api Alert Rules

Args: none **Returns:** JSON api_alert_rules() · L4289

GET `/api/alert-thresholds`

Api Alert Thresholds

Args: none **Returns:** JSON api_alert_thresholds() · L4205

GET `/api/alert_dashboard`

Api Alert Dashboard

Args: `days: int = 7; server: str = None; severity: str = None` **Returns:** JSON api_alert_dashboard() · L3743

GET `/api/alerts/incidents`

List alert incidents. Filter by status (open|resolved), risk_level, server.

Args: `status: str = None; risk_level: str = None; server: str = None; limit: int = 500` **Returns:** JSON api_alert_incidents() · L5439

POST `/api/alerts/incidents/{incident_id}/resolve`

Manually resolve an open incident. ONLY a valid, active customer user may do this, and a description is required — both enforced in alerts.resolve_incident(). Body: {"resolved_by": "user@customer.com", "description": "what was done"}.

Args: `incident_id: int` **Returns:** JSON api_resolve_incident() · L6111

POST `/api/alerts/incidents/{incident_id}/resolve-ui`

Resolve an open incident with resolver name, resolution type, an optional resolved-at timestamp, and a description — the same capability as the Open Alerts dashboard resolve form, exposed as JSON. resolved_by is stored as-is (e.g. the Grafana username); the gate is the caller's own authentication, not web.users. Body: {"resolved_by": "...", "resolution_type": "Fixed", "resolved_at": "2026-07-15T14:30" (optional; ISO or 'YYYY-MM-DD HH:MM'; default now), "resolution_description": "..."}. Calls alerts.resolve_incident_ui (migration 6620).

Args: `incident_id: int` **Returns:** JSON api_resolve_incident_ui() · L6143

GET `/api/alerts/lifecycle-config`

Auto-resolve timeouts per severity (the 'time configured' the customer sets).

Args: none **Returns:** JSON api_get_lifecycle_config() · L6181

POST `/api/alerts/lifecycle-config`

Update the auto-resolve timeout for a severity. Body: {"risk_level": "high", "auto_resolve_enabled": true, "auto_resolve_after_hours": 48}.

Args: none **Returns:** JSON api_set_lifecycle_config() · L6200

GET `/api/alerts/log/{alert_id}`

Active Detection Findings for one alert (dashboard panel 455) as JSON — i.e. SELECT * FROM monitoring.get_alert_log_resultset_byid(alert_id), with an optional entry_date window. Each finding's `result` json is expanded to a dict (matching the dashboard's field extraction). Path: alert_id (the alert_log row/alert_id). Query params: from, to (Grafana epoch-ms or ISO local; both optional).

Args: `alert_id: str` **Returns:** JSON api_alert_log_detail() · L5646

GET `/api/alerts/open`

The Open Alerts feed (dashboard panel 120) as JSON — one row per server+root_cause (latest), joined to server name and root-cause metadata, with mail-sent and blocked timestamps. Query params (all optional): from, to (Grafana epoch-ms or ISO local), risk_level (critical|high|medium|low), server, root_cause_id, status (open|resolved), limit (default 500).

Args: none **Returns:** JSON api_alerts_open() · L5575

GET `/api/alerts/ransomware`

Ransomware Guard feed (dashboard 'ransomware-guard' panel 20) as JSON: SEC-SQL-AUD-031-* alerts with server, risk, login, session id, the root-cause description and a truncated sql_text. Query params (all optional): from, to (Grafana epoch-ms or ISO local; when neither is given, defaults to the last 30 days), server, risk_level, root_cause_id (defaults to the SEC-SQL-AUD-031-% family), limit (default 500).

Args: none **Returns:** JSON api_alerts_ransomware() · L5682

GET `/api/alerts/summary`

Open Alerts dashboard header counts as JSON (panels 456/101/102/105/104/103). Query params: from, to (Grafana epoch-ms or ISO local; both optional). Returns open total + per-severity open counts (SEC-* root causes) and the number of monitored servers.

Args: none **Returns:** JSON api_alerts_summary() · L5539

POST `/api/delete-alert-rule`

Api Delete Alert Rule

Args: none **Returns:** JSON api_delete_alert_rule() · L4342

POST `/api/delete-global-param`

Api Delete Global Param

Args: none **Returns:** JSON api_delete_global_param() · L4397

POST `/api/delete-threshold`

Api Delete Threshold

Args: none **Returns:** JSON api_delete_threshold() · L4260

POST `/api/delete-webhook-alert`

Api Delete Webhook Alert

Args: none **Returns:** JSON api_delete_webhook_alert() · L4463

GET `/api/global-params`

Api Global Params

Args: none **Returns:** JSON api_global_params() · L4366

GET `/api/grc-alerts/channels`

Api Get Alert Channels

Args: none **Returns:** JSON api_get_alert_channels() · L7106

POST `/api/grc-alerts/channels`

Api Create Alert Channel

Args: none **Returns:** JSON api_create_alert_channel() · L7120

DELETE `/api/grc-alerts/channels/{channel_id}`

Api Delete Alert Channel

Args: `channel_id: int` **Returns:** JSON api_delete_alert_channel() · L7167

PUT `/api/grc-alerts/channels/{channel_id}`

Api Update Alert Channel

Args: `channel_id: int` **Returns:** JSON api_update_alert_channel() · L7143

GET `/api/grc-alerts/delivery-log`

Api Grc Alert Delivery Log

Args: none **Returns:** JSON api_grc_alert_delivery_log() · L7181

GET `/api/webhook-alerts`

Api Webhook Alerts

Args: none **Returns:** JSON api_webhook_alerts() · L4421

App Login Guard (8)

GET `/api/app-logins`

Api App Logins List

Args: none **Returns:** JSON api_app_logins_list() · L1346

POST `/api/app-logins/add`

Api App Logins Add

Args: none **Returns:** JSON api_app_logins_add() · L1354

POST `/api/app-logins/delete`

Api App Logins Delete

Args: none **Returns:** JSON api_app_logins_delete() · L1378

POST `/api/app-logins/set`

Api App Logins Set

Args: none **Returns:** JSON api_app_logins_set() · L1368

GET `/api/programs`

Api Programs List

Args: none **Returns:** JSON api_programs_list() · L1388

POST `/api/programs/add`

Api Programs Add

Args: none **Returns:** JSON api_programs_add() · L1396

POST `/api/programs/delete`

Api Programs Delete

Args: none **Returns:** JSON api_programs_delete() · L1420

POST `/api/programs/set`

Api Programs Set

Args: none **Returns:** JSON api_programs_set() · L1410

Asset Discovery (2)

GET `/api/discovery-candidates`

Api Discovery Candidates

Args: none **Returns:** JSON api_discovery_candidates() · L7611

PUT `/api/discovery-candidates/{candidate_id}`

Api Update Discovery Candidate

Args: `candidate_id: int` **Returns:** JSON api_update_discovery_candidate() · L7634

Blocker Activity (3)

GET `/api/blocker/blocks`

Executed blocks (alerts.blocks, panel 11).

Args: none **Returns:** JSON api_blocker_blocks() · L6049

GET `/api/blocker/log`

Blocker activity log (alerts.blocker_log, panel 10).

Args: none **Returns:** JSON api_blocker_log() · L6034

GET `/api/blocker/summary`

Blocker action counts (killed/dry-run/skipped/errors) + current dry-run mode (panels 3-7).

Args: none **Returns:** JSON api_blocker_summary() · L6008

Compliance Reporting (10)

GET `/api/audit-log`

Api Audit Log

Args: `page: int = 1; page_size: int = 50; action: str = None; server_name: str = None; db_user: str = None; from_dt: str = None; to_dt: str = None` **Returns:** JSON api_audit_log() · L5320

GET `/api/compliance-reports`

List generated PDF files in the reports/compliance directory.

Args: none **Returns:** JSON api_list_reports() · L6237

GET `/api/compliance-reports/download/{filename}`

Api Download Report

Args: `filename: str` **Returns:** JSON api_download_report() · L6261

POST `/api/compliance-reports/run`

Api Run Report

Args: none **Returns:** JSON api_run_report() · L6271

GET `/api/compliance-reports/schedules`

Api Get Schedules

Args: none **Returns:** JSON api_get_schedules() · L5381

PUT `/api/compliance-reports/schedules/{schedule_id}`

Api Update Schedule

Args: `schedule_id: int` **Returns:** JSON api_update_schedule() · L5407

GET `/api/regulation-controls`

Api Regulation Controls

Args: `regulation: str = None; status: str = None` **Returns:** JSON api_regulation_controls() · L7305

GET `/api/report-schedules`

Return all reports and their schedule configurations.

Args: none **Returns:** JSON api_report_schedules() · L3869

GET `/api/reports/list`

Api Reports List

Args: none **Returns:** JSON api_reports_list() · L2559

POST `/api/toggle-report`

Toggle a report's active status.

Args: none **Returns:** JSON api_toggle_report() · L4011

DDL Audit (2)

GET `/api/ddl-audit/logs`

Api Ddl Audit Logs

Args: `ddl_command: str = None; risk_level: str = None; regulation: str = None` **Returns:** JSON api_ddl_audit_logs() · L6871

GET `/api/ddl-audit/summary`

Api Ddl Audit Summary

Args: none **Returns:** JSON api_ddl_audit_summary() · L6902

Data Protection & Privacy (21)

GET `/api/data-protection/unmasked`

Data Protection - Unmasked sensitive data as JSON: the contents of monitoring.v_SEC_SQL_PRI_001_RC10 (SEC-SQL-PRI-001-RC10 detection view). Query params: server (optional exact match, applied only if the view exposes a 'server' column), limit (default 500).

Args: none **Returns:** JSON api_data_protection_unmasked() · L5769

GET `/api/masking-rules`

Api Get Masking Rules

Args: `pii_type: str = None; mask_type: str = None; is_active: str = None; server_name: str = None` **Returns:** JSON api_get_masking_rules() · L6359

POST `/api/masking-rules`

Api Create Masking Rule

Args: none **Returns:** JSON api_create_masking_rule() · L6400

POST `/api/masking-rules/sync`

Trigger an immediate sync from monitoring.sensitive_schema.

Args: none **Returns:** JSON api_sync_masking_rules() · L6466

PUT `/api/masking-rules/{rule_id}`

Api Update Masking Rule

Args: `rule_id: int` **Returns:** JSON api_update_masking_rule() · L6437

GET `/api/retention/dump-location`

Current dump/archive location (panel 4).

Args: none **Returns:** JSON api_retention_dump_location() · L5999

GET `/api/retention/dump-metrics`

Configured per-metric dump retention (config.dump_metrics, panel 5).

Args: none **Returns:** JSON api_retention_dump_metrics() · L5985

GET `/api/retention/dumps`

Archived dumps catalog (config.dumps, panel 8).

Args: none **Returns:** JSON api_retention_dumps() · L5992

GET `/api/retention/executions`

Api Retention Executions

Args: none **Returns:** JSON api_retention_executions() · L7727

GET `/api/retention/overview`

Retention overview per category/item with size, row count and configured dump retention months (panel 2).

Args: none **Returns:** JSON api_retention_overview() · L5972

GET `/api/retention/policies`

Api Retention Policies

Args: none **Returns:** JSON api_retention_policies() · L7663

POST `/api/retention/policies`

Api Create Retention Policy

Args: none **Returns:** JSON api_create_retention_policy() · L7679

PUT `/api/retention/policies/{retention_id}`

Api Update Retention Policy

Args: `retention_id: int` **Returns:** JSON api_update_retention_policy() · L7704

GET `/api/sensitive-schema`

Api Sensitive Schema

Args: none **Returns:** JSON api_sensitive_schema() · L4058

POST `/api/sensitive-schema/bulk-toggle`

Api Bulk Toggle Sensitive
Args: none **Returns:** JSON api_bulk_toggle_sensitive() · L4140

POST `/api/sensitive-schema/discover`

Api Discover Sensitive
Args: none **Returns:** JSON api_discover_sensitive() · L4162

POST `/api/sensitive-schema/toggle`

Api Toggle Sensitive
Args: none **Returns:** JSON api_toggle_sensitive() · L4116

GET `/api/tls/policies`

Api Tls Policies
Args: none **Returns:** JSON api_tls_policies() · L7440

POST `/api/tls/policies`

Api Create Tls Policy
Args: none **Returns:** JSON api_create_tls_policy() · L7454

PUT `/api/tls/policies/{tls_policy_id}`

Api Update Tls Policy
Args: `tls_policy_id: int` **Returns:** JSON api_update_tls_policy() · L7476

GET `/api/tls/violations`

Api Tls Violations
Args: none **Returns:** JSON api_tls_violations() · L7418

Detection Catalogs (2)

GET `/api/database-restore`

Database restore catalog as JSON: SEC-domain root causes in the AZ/CFG/ENC/VS areas (area, issue, id, name, description) from rootcause.v_rootcauses. Query params: area_code (optional CSV; default 'AZ,CFG,ENC,VS'), root_cause_id (optional exact match).
Args: none **Returns:** JSON api_database_restore() · L5805

GET `/api/policy-enforcement-and-protection`

Policy Enforcement & Protection catalog as JSON: distinct SEC-domain root causes (id, name, description) from rootcause.v_rootcauses. Query params: root_cause_id (optional exact match), vendor_name (default sqlserver). Note: this returns only the customer-facing name/description of each detection, not the underlying detection logic.
Args: none **Returns:** JSON api_policy_enforcement() · L5738

Detections (2)

GET `/api/same-login-multihost/findings`

Detailed findings for the same-login-from-multiple-hosts detection (monitoring.v_sec_sql_acc_010_rc10, panel 12).
Args: none **Returns:** JSON api_same_login_findings() · L6064

GET `/api/same-login-multihost/root-causes`

Root cause(s) behind the same-login-multi-host detection (panel 16).
Args: none **Returns:** JSON api_same_login_root_causes() · L6073

Email & Notifications (6)

POST `/api/email-panel`

Generate the panel PDF and email it to the supplied recipients. Reads SMTP details from config.mail_config.
Args: none **Returns:** HTMLResponse email_panel() · L4853

GET `/api/email-panel-form`

Render a small HTML dialog form prompting for email recipients. All incoming query params (dashboard, panel, from, to, and every Grafana variable) are preserved as hidden inputs, so submitting the form posts everything needed to /api/email-panel.
Args: none **Returns:** HTMLResponse email_panel_form() · L4780

POST `/api/mail-config/delete`

Api Delete Mail Config
Args: none **Returns:** JSON api_delete_mail_config() · L2322

GET `/api/mail-config/list`

Api List Mail Config
Args: none **Returns:** JSON api_list_mail_config() · L2725

POST `/api/mail-group/delete`

Api Delete Mail Group
Args: none **Returns:** JSON api_delete_mail_group() · L2339

GET `/api/print-panel`

Generate a PDF report for a Grafana table panel. Required query params: dashboard: dashboard uid (e.g. ad7kkx7) panel: numeric panel id All other query params are treated as Grafana template variable values (e.g. ?server=foo&risk_level=high). Variables found in the panel's SQL but not in the request stay as $name and will likely fail at execute time — pass everything the panel needs.
Args: none **Returns:** JSON print_panel() · L4740

Exclusions (4)

GET `/api/exclude-logins`

All rows of metrics.exclude_logins for the /exclude_logins config page.
Args: none **Returns:** JSON api_exclude_logins_list() · L1118

POST `/api/exclude-logins/add`

Api Exclude Logins Add
Args: none **Returns:** JSON api_exclude_logins_add() · L1136

POST `/api/exclude-logins/delete`

Api Exclude Logins Delete
Args: none **Returns:** JSON api_exclude_logins_delete() · L1169

POST `/api/exclude-logins/set`

Api Exclude Logins Set
Args: none **Returns:** JSON api_exclude_logins_set() · L1152

Firewall Policies (5)

GET `/api/firewall-policies`

Api Get Policies
Args: `action: str = None; regulation: str = None; is_active: str = None` **Returns:** JSON api_get_policies() · L5118

POST `/api/firewall-policies`

Api Create Policy
Args: none **Returns:** JSON api_create_policy() · L5157

POST `/api/firewall-policies/apply-template`

Insert the default seed policies for a regulation (skips conflicts).
Args: none **Returns:** JSON api_apply_template() · L5243

DELETE `/api/firewall-policies/{policy_id}`

Api Delete Policy
Args: `policy_id: int` **Returns:** JSON api_delete_policy() · L5224

PUT `/api/firewall-policies/{policy_id}`

Api Update Policy
Args: `policy_id: int` **Returns:** JSON api_update_policy() · L5195

Fleet (1)

GET `/api/fleet`

Api Fleet
Args: `days: int = 7` **Returns:** JSON api_fleet() · L3820

Governance, Risk & Compliance (GRC) (19)

GET `/api/access-review/cycles`

Api Access Review Cycles
Args: none **Returns:** JSON api_access_review_cycles() · L7370

POST `/api/access-review/cycles`

Api Create Access Review Cycle
Args: none **Returns:** JSON api_create_access_review_cycle() · L7386

GET `/api/access-review/instances`

Api Access Review Instances
Args: none **Returns:** JSON api_access_review_instances() · L7344

GET `/api/evidence-packages`

Api Get Evidence Packages

Args: none **Returns:** JSON api_get_evidence_packages() · L7759

POST `/api/evidence-packages/generate`

Api Generate Evidence Package

Args: none **Returns:** JSON api_generate_evidence_package() · L7783

GET `/api/evidence-packages/{package_id}/download/{file_type}`

Api Download Evidence Package

Args: `package_id: int; file_type: str` **Returns:** JSON api_download_evidence_package() · L7806

GET `/api/policy-exceptions`

Api Get Policy Exceptions

Args: `status: str = None` **Returns:** JSON api_get_policy_exceptions() · L7212

POST `/api/policy-exceptions`

Api Create Policy Exception

Args: none **Returns:** JSON api_create_policy_exception() · L7243

PUT `/api/policy-exceptions/{exception_id}`

Api Update Policy Exception

Args: `exception_id: int` **Returns:** JSON api_update_policy_exception() · L7267

GET `/api/privilege-changes`

Api Privilege Changes

Args: `operation: str = None; regulation: str = None` **Returns:** JSON api_privilege_changes() · L6936

GET `/api/privilege-changes/recent-summary`

Api Privilege Recent Summary

Args: none **Returns:** JSON api_privilege_recent_summary() · L6965

GET `/api/risk-level/list`

Api List Risk Levels

Args: none **Returns:** JSON api_list_risk_levels() · L2711

GET `/api/risk-scores`

Api Risk Scores

Args: `min_risk_score: int = 0; server_name: str = None; login_name: str = None` **Returns:** JSON api_risk_scores() · L6292

GET `/api/risk-scores/{server_name}/{login_name}/events`

Api Risk Events

Args: `server_name: str; login_name: str` **Returns:** JSON api_risk_events() · L6330

GET `/api/sod/rules`

Api Sod Rules

Args: none **Returns:** JSON api_sod_rules() · L7038

POST `/api/sod/rules`

Api Sod Create Rule

Args: none **Returns:** JSON api_sod_create_rule() · L7052

PUT `/api/sod/rules/{rule_id}`

Api Sod Update Rule

Args: `rule_id: int` **Returns:** JSON api_sod_update_rule() · L7074

GET `/api/sod/violations`

Api Sod Violations

Args: none **Returns:** JSON api_sod_violations() · L6999

POST `/api/sod/violations/{violation_id}/acknowledge`

Api Sod Acknowledge

Args: `violation_id: int` **Returns:** JSON api_sod_acknowledge() · L7021

IPS Report (8)

GET `/api/ips/blocked`

Blocked intrusions by attack/type/severity (panel 10).

Args: none **Returns:** JSON api_ips_blocked() · L5919

GET `/api/ips/by-severity`

Intrusions by severity (panel 7).

Args: none **Returns:** JSON api_ips_by_severity() · L5870

GET `/api/ips/by-type`

Intrusions by type (panel 8).
Args: none **Returns:** JSON api_ips_by_type() · L5882

GET `/api/ips/monitored`

Monitored intrusions by attack/type (panel 11).
Args: none **Returns:** JSON api_ips_monitored() · L5907

GET `/api/ips/summary`

IPS header stats: total / critical / high / medium intrusion events, blocked (FortiAnalyzer IPS), and monitored (mailed) — panels 1-6.
Args: none **Returns:** JSON api_ips_summary() · L5841

GET `/api/ips/timeline`

Intrusion events timeline, hourly buckets by severity (panel 9).
Args: none **Returns:** JSON api_ips_timeline() · L5895

GET `/api/ips/top-sources`

Top attack sources as % of total, with the matched metric metadata (panel 12).
Args: none **Returns:** JSON api_ips_top_sources() · L5947

GET `/api/ips/top-victims`

Top victims (servers) by attack/type (panel 13).
Args: none **Returns:** JSON api_ips_top_victims() · L5934

Performance (1)

GET `/api/performance/stored-proc-slow`

Stored procedures running slower than their historical average (SEC-SQL-QE-001-RC01; monitoring.v_sec_sql_qe_001_rc01): procedure, database, login, avg vs actual duration. Query params: from, to (epoch-ms or ISO local; default last 7 days), server (optional), limit (default 500).
Args: none **Returns:** JSON api_stored_proc_slow() · L6088

Resolve Incident (1)

POST `/api/resolve-incident`

Resolve Incident Submit
Args: none **Returns:** HTMLResponse resolve_incident_submit() · L5028

Resolve Incident Form (1)

GET `/api/resolve-incident-form`

Resolve Incident Form
Args: none **Returns:** HTMLResponse resolve_incident_form() · L4932

SIEM Integration (5)

GET `/api/siem/ips-dashboard`

Api Siem Ips Dashboard
Args: none **Returns:** JSON api_siem_ips_dashboard() · L7847

GET `/api/siem/ips-report/download`

Api Siem Ips Download
Args: none **Returns:** JSON api_siem_ips_download() · L7969

POST `/api/siem/ips-report/email`

Api Siem Ips Email
Args: none **Returns:** JSON api_siem_ips_email() · L7927

GET `/api/siem/ips-report/schedule`

Api Siem Ips Schedule Get
Args: none **Returns:** JSON api_siem_ips_schedule_get() · L7984

POST `/api/siem/ips-report/schedule`

Api Siem Ips Schedule Post
Args: none **Returns:** JSON api_siem_ips_schedule_post() · L8023

Threat Response (13)

GET `/api/threat-response/blocked-ips`

Api Get Blocked Ips

Args: none **Returns:** JSON api_get_blocked_ips() · L6589

POST `/api/threat-response/blocked-ips`

Api Block Ip

Args: none **Returns:** JSON api_block_ip() · L6615

DELETE `/api/threat-response/blocked-ips/{block_id}`

Api Release Ip

Args: `block_id: int` **Returns:** JSON api_release_ip() · L6647

GET `/api/threat-response/incidents`

Api Get Incidents

Args: `status: str = None; severity: str = None` **Returns:** JSON api_get_incidents() · L6491

POST `/api/threat-response/incidents`

Api Create Incident

Args: none **Returns:** JSON api_create_incident() · L6529

PUT `/api/threat-response/incidents/{incident_id}`

Api Update Incident

Args: `incident_id: int` **Returns:** JSON api_update_incident() · L6558

GET `/api/threat-response/playbooks`

Api Get Playbooks

Args: none **Returns:** JSON api_get_playbooks() · L6749

POST `/api/threat-response/playbooks`

Api Create Playbook

Args: none **Returns:** JSON api_create_playbook() · L6770

PUT `/api/threat-response/playbooks/{playbook_id}`

Api Update Playbook

Args: `playbook_id: int` **Returns:** JSON api_update_playbook() · L6804

GET `/api/threat-response/response-log`

Api Get Response Log

Args: none **Returns:** JSON api_get_response_log() · L6836

GET `/api/threat-response/suspended-users`

Api Get Suspended Users

Args: none **Returns:** JSON api_get_suspended_users() · L6668

POST `/api/threat-response/suspended-users`

Api Suspend User

Args: none **Returns:** JSON api_suspend_user() · L6695

DELETE `/api/threat-response/suspended-users/{suspension_id}`

Api Release User

Args: `suspension_id: int` **Returns:** JSON api_release_user() · L6728

Vulnerability Management (5)

GET `/api/vulnerability/cve-watchlist`

Api Cve Watchlist

Args: none **Returns:** JSON api_cve_watchlist() · L7532

POST `/api/vulnerability/cve-watchlist`

Api Create Cve

Args: none **Returns:** JSON api_create_cve() · L7548

PUT `/api/vulnerability/cve-watchlist/{cve_id}`

Api Update Cve

Args: `cve_id: str` **Returns:** JSON api_update_cve() · L7574

GET `/api/vulnerability/findings`

Api Vulnerability Findings

Args: none **Returns:** JSON api_vulnerability_findings() · L7508

POST `/api/vulnerability/scan`

Api Trigger Vulnerability Scan
Args: none **Returns:** JSON api_trigger_vulnerability_scan() · L7596

Web UI Pages (HTML) (75)

GET `/`

Dbdome Main
Args: none **Returns:** RedirectResponse dbdome_main() · L175

GET `/addrecipients`

Addrecipients
Args: none **Returns:** HTMLResponse addrecipients() · L215

GET `/alert_dashboard`

Alert Dashboard
Args: none **Returns:** HTMLResponse alert_dashboard() · L3737

GET `/alert_rules`

Alert Rules Page
Args: none **Returns:** HTMLResponse alert_rules_page() · L4284

GET `/alert_thresholds`

Alert Thresholds Page
Args: none **Returns:** HTMLResponse alert_thresholds_page() · L4200

GET `/app_login_guard`

Config page for the app_login_guard watchlists (metrics.app_logins + metrics.programs), mirroring the Excluded Logins page. Opened from a dashboard button. Entries are case-insensitive LIKE patterns; server '%' = all servers.
Args: none **Returns:** HTMLResponse app_login_guard_page() · L1430

GET `/capture_report/{reportname}`

Capture Report
Args: `reportname: str` **Returns:** HTMLResponse capture_report() · L620

GET `/custom_metrics`

Serverform Page
Args: none **Returns:** HTMLResponse serverform_page() · L203

GET `/custom_metrics_update`

Submit
Args: none **Returns:** HTML submit() · L3105

GET `/db_restore`

Revert ALL changes by restoring the database from its backup (reverts static masking / anonymization, and everything else). ?server=&db=&backup=&referer= backup optional - latest full backup of the DB is used when omitted.
Args: none **Returns:** HTMLResponse db_restore() · L1806

GET `/dbdome`

Dbdome Main
Args: none **Returns:** RedirectResponse dbdome_main() · L165

GET `/download_report/{reportname}`

Download Report
Args: `reportname: str` **Returns:** HTMLResponse download_report() · L867

GET `/dp_anonymize`

Anonymization (non-prod, IRREVERSIBLE). technique: suppress | substitute | generalize | shuffle | hash | realistic. If 'column' is omitted, EVERY big character column (varchar/nvarchar/char/nchar >= min_len chars, or MAX) in the table is anonymized. <table> carries the schema (e.g. 'dbo.Cards'); no separate schema param. ?server=&db=&table=&[column=]&[technique=]&[min_len=]&referer=
Args: none **Returns:** HTMLResponse dp_anonymize() · L1597

GET `/dp_anonymize_actual`

Staged anonymization phase 2: switch the live table with its latest pre-copy. ?server=&db=&schema=&table=&referer= (table may be 'schema.table')
Args: none **Returns:** HTMLResponse dp_anonymize_actual() · L1644

GET `/dp_anonymize_confirm`

Staged anonymization phase 3: create the backup schema (default dbdome_backup) if missing and move the original (dated copy) into it. ?server=&db=&schema=&table=&[backup_schema=]&referer= (table may be 'schema.table')
Args: none **Returns:** HTMLResponse dp_anonymize_confirm() · L1660

GET `/dp_anonymize_pre`

Staged anonymization phase 1: copy <schema>.<table> to <table>_<datetime> and anonymize the COPY (all big varchars; original untouched). ?server=&db=&schema=&table=&[technique=]&[min_len=]&referer= (table may be 'schema.table')

Args: none **Returns:** HTMLResponse dp_anonymize_pre() · L1622

GET `/dp_anonymize_preview`

Preview the anonymized dry-run copy (latest <table>_<datetime>, or ?copy=). Read-only SELECT TOP <rows> *, rendered as an HTML table. ?server=&db=&table=&[copy=]&[rows=50] (table may be '<schema>.<table>')

Args: none **Returns:** HTMLResponse dp_anonymize_preview() · L1678

GET `/dp_dynamic_mask`

Data protection - Dynamic Data Masking. Caller selects server, database, table, column and the mask function (default()/email()/random(a,b)/ partial(p,"pad",s)); the column is masked on display. Audited in metrics.dp_log. <table> carries the schema (e.g. 'dbo.Cards'); no separate schema param. Dashboard link: ?server=&db=&table=&column=&function=&referer=

Args: none **Returns:** HTMLResponse dp_dynamic_mask() · L1535

GET `/dp_revert_mask`

Revert Dynamic Data Masking on a column (DROP MASKED). <table> carries the schema (e.g. 'dbo.Cards'); no separate schema param. ?server=&db=&table=&column=&referer=

Args: none **Returns:** HTMLResponse dp_revert_mask() · L1772

GET `/dp_revert_tokenize`

Revert tokenization: de-tokenize a column from the vault. <table> carries the schema (e.g. 'dbo.Cards'); no separate schema param. ?server=&db=&table=&column=&referer=

Args: none **Returns:** HTMLResponse dp_revert_tokenize() · L1789

GET `/dp_static_mask`

Static masking (non-prod, DESTRUCTIVE). Caller selects column + function: null | default | fixed:<v> | shuffle | partial(p,"pad",s). <table> carries the schema (e.g. 'dbo.Cards'); no separate schema param. ?server=&db=&table=&column=&function=&referer=

Args: none **Returns:** HTMLResponse dp_static_mask() · L1578

GET `/dp_tokenize`

Tokenization (production, §4.2). Caller selects column + encryption type: passphrase | aes256. Real value -> encrypted TokenVault; column holds a token. <table> carries the schema (e.g. 'dbo.Cards'); no separate schema param. ?server=&db=&table=&column=&encryption=&referer=

Args: none **Returns:** HTMLResponse dp_tokenize() · L1715

GET `/dp_tokenize_actual`

Staged tokenization phase 2: switch the live table with its latest tokenize_pre copy. <table> carries the schema (e.g. 'dbo.Cards'); no separate schema param. ?server=&db=&table=&referer=

Args: none **Returns:** HTMLResponse dp_tokenize_actual() · L1754

GET `/dp_tokenize_pre`

Staged tokenization phase 1: copy <table> to <table>_<datetime> and tokenize <column> on the COPY (original untouched); copy mirrored to Postgres. <table> carries the schema (e.g. 'dbo.Cards'); no separate schema param. ?server=&db=&table=&column=&[encryption=]&referer=

Args: none **Returns:** HTMLResponse dp_tokenize_pre() · L1734

GET `/email_configuration`

Serverform Page

Args: none **Returns:** HTMLResponse serverform_page() · L2923

GET `/encrypt_column`

Backup the table to [unencrypted] and tokenize the column. Authorisation + masking-style dry-run gate (config.global_params 'encryption_dry_run').

Args: none **Returns:** HTMLResponse encrypt_column() · L2184

GET `/exclude_login_set`

Toggle metrics.exclude_logins.is_active via set_exclude_login, then redirect back to the referring dashboard (same pattern as /webook_alert_set).

Args: none **Returns:** HTMLResponse exclude_login_set() · L1089

GET `/exclude_logins`

Config page for metrics.exclude_logins (opened from the Excluded Logins dashboard button): list, add, enable/disable and delete excluded logins via the /api/exclude-logins endpoints.

Args: none **Returns:** HTMLResponse exclude_logins_page() · L1185

GET `/fleet`

Fleet Dashboard

Args: none **Returns:** HTMLResponse fleet_dashboard() · L3814

GET `/generate_report/{reportname}`

Capture Report
Args: `reportname: str` **Returns:** HTMLResponse capture_report() · L702

GET `/global_param_set`

Set a whitelisted config.global_params key via set_global_param, then redirect back to the referring dashboard (same pattern as /webook_alert_set).
Args: none **Returns:** HTMLResponse global_param_set() · L1063

GET `/global_params`

Global Params Page
Args: none **Returns:** HTMLResponse global_params_page() · L4361

GET `/grc/access-review`

Grc Access Review Page
Args: none **Returns:** HTMLResponse grc_access_review_page() · L7339

GET `/grc/alert-channels`

Grc Alert Channels Page
Args: none **Returns:** HTMLResponse grc_alert_channels_page() · L7101

GET `/grc/audit-log`

Grc Audit Log Page
Args: none **Returns:** HTMLResponse grc_audit_log_page() · L5095

GET `/grc/compliance-reports`

Grc Compliance Reports Page
Args: none **Returns:** HTMLResponse grc_compliance_reports_page() · L5100

GET `/grc/data-retention`

Grc Data Retention Page
Args: none **Returns:** HTMLResponse grc_data_retention_page() · L7658

GET `/grc/ddl-audit`

Grc Ddl Audit Page
Args: none **Returns:** HTMLResponse grc_ddl_audit_page() · L6866

GET `/grc/evidence-packages`

Grc Evidence Packages Page
Args: none **Returns:** HTMLResponse grc_evidence_packages_page() · L7754

GET `/grc/masking-rules`

Grc Masking Rules Page
Args: none **Returns:** HTMLResponse grc_masking_rules_page() · L5110

GET `/grc/policies`

Grc Policies Page
Args: none **Returns:** HTMLResponse grc_policies_page() · L5090

GET `/grc/policy-exceptions`

Grc Policy Exceptions Page
Args: none **Returns:** HTMLResponse grc_policy_exceptions_page() · L7207

GET `/grc/privilege-changes`

Grc Privilege Changes Page
Args: none **Returns:** HTMLResponse grc_privilege_changes_page() · L6931

GET `/grc/regulation-controls`

Grc Regulation Controls Page
Args: none **Returns:** HTMLResponse grc_regulation_controls_page() · L7300

GET `/grc/risk-dashboard`

Grc Risk Dashboard Page
Args: none **Returns:** HTMLResponse grc_risk_dashboard_page() · L5105

GET `/grc/sod-violations`

Grc Sod Violations Page
Args: none **Returns:** HTMLResponse grc_sod_violations_page() · L6994

GET `/grc/threat-response`

Grc Threat Response Page
Args: none **Returns:** HTMLResponse grc_threat_response_page() · L6483

GET `/grc/tls-enforcement`

Grc Tls Enforcement Page

Args: none **Returns:** HTMLResponse grc_tls_enforcement_page() · L7413

GET /grc/vulnerability-assessment

Grc Vulnerability Page

Args: none **Returns:** HTMLResponse grc_vulnerability_page() · L7503

GET /mailconfiguration

Addrecipients

Args: none **Returns:** HTMLResponse addrecipients() · L346

GET /mask_column

Create SQL Server Dynamic Data Masking on a discovered sensitive column, then redirect back to the Sensitive Columns Explorer (same pattern as /sensitive_column_add). Authorisation: the (server,db,schema,table,column) must be a column DBDOME flagged as sensitive. Gated by the config.global_params 'masking_dry_run' switch (default true = log only).

Args: none **Returns:** HTMLResponse mask_column() · L1994

GET /metric_enable

Metric Enable

Args: none **Returns:** HTML metric_enable() · L2934

GET /metric_enable_all_servers

Metric Enable

Args: none **Returns:** HTML metric_enable() · L2975

GET /processform

Processform Page

Args: none **Returns:** HTMLResponse processform_page() · L186

GET /rbac_provision

RBAC provisioning: for the selected SQL Server + database, back up the database (original name + unix time) then create the standard role set with least-privilege grants (db_readonly/db_dataentry/db_manager/db_report_viewer) and the server role srv_dba (GRANT ALTER ANY CONNECTION). Each step is audited in metrics.rbac_log. Triggered from a dashboard link: ?server=&db=&referer=

Args: none **Returns:** HTMLResponse rbac_provision() · L1507

GET /report_schedule

Report Schedule Page

Args: none **Returns:** HTMLResponse report_schedule_page() · L3863

GET /reportbymail/{reportname}

Show Send Report By Mail

Args: `reportname: str` **Returns:** HTMLResponse show_send_report_by_mail() · L784

GET /reports

Reports List Page

Args: none **Returns:** HTMLResponse reports_list_page() · L2398

GET /reports/edit

Reports Edit Page

Args: none **Returns:** HTMLResponse reports_edit_page() · L2424

GET /retention_policy

Serverform Page

Args: none **Returns:** HTMLResponse serverform_page() · L2929

GET /revert_encryption

Drop the tokenized table and restore its [unencrypted] backup to the original schema/name.

Args: none **Returns:** HTMLResponse revert_encryption() · L2229

GET /risklevel

Risklevel Page

Args: none **Returns:** HTMLResponse risklevel_page() · L2564

GET /risklevel_toggle

Toggle (or set) is_active on a single rootcause.risk_level row. Query params: risk_level (required) — name e.g. 'low', 'medium', 'high', 'critical' action (optional) — 'enable' | 'disable' | 'toggle' (default: toggle) sender (optional) — Grafana dashboard return path

Args: none **Returns:** HTML risklevel_toggle() · L2652

GET /root_cause_active

Enable/disable a root cause (rootcause.resolution_paths.is_active) via rootcause.set_root_cause_active, then redirect back to the referring dashboard (same pattern as /webook_alert_set).

Args: none **Returns:** HTMLResponse root_cause_active() · L1823

GET `/root_cause_risk`

Set a root cause's risk level (rootcause.resolution_paths.risk_level) via rootcause.set_root_cause_risk (validates low/medium/high/critical), then redirect back to the referring dashboard (same pattern as /webook_alert_set).

Args: none **Returns:** HTMLResponse root_cause_risk() · L1850

GET `/sensitive_column_add`

Add a column to metrics.sensitive_columns via add_sensitive_column, then redirect back to the referring dashboard (same pattern as /webook_alert_set).

Args: none **Returns:** HTMLResponse sensitive_column_add() · L1033

GET `/sensitive_schema`

Sensitive Schema Page

Args: none **Returns:** HTMLResponse sensitive_schema_page() · L4052

GET `/server_enable`

Server Enable

Args: none **Returns:** HTMLResponse server_enable() · L957

GET `/serverform`

Serverform Page

Args: none **Returns:** HTMLResponse serverform_page() · L191

GET `/siem/ips-dashboard`

Siem Ips Dashboard Page

Args: none **Returns:** HTMLResponse siem_ips_dashboard_page() · L7841

GET `/siem_configuration`

Siem Configuration

Args: none **Returns:** HTMLResponse siem_configuration() · L197

GET `/take_action`

Submit

Args: `row_id: int` **Returns:** HTML submit() · L3126

GET `/update_schema_column`

Update Schema Column

Args: none **Returns:** HTMLResponse update_schema_column() · L3022

GET `/webhook_alerts`

Webhook Alerts Page

Args: none **Returns:** HTMLResponse webhook_alerts_page() · L4416

GET `/webook_alert_set`

Toggle a config.webook_alerts flag via the set_webook_alert procedure, then redirect back to the referring dashboard (same pattern as /server_enable).

Args: none **Returns:** HTMLResponse webook_alert_set() · L1003

Web Actions & Form Submissions (23)

POST `/reports/save`

Reports Save

Args: none **Returns:** HTML reports_save() · L2504

POST `/submit-addrecipients`

Addrecipients

Args: `mail_config_id: str = Form(PydanticUndefined); group_name: str = Form(PydanticUndefined); recipients: str = Form(PydanticUndefined); referer: str = Form(PydanticUndefined); row_id: str = Form(None); is_active: bool = Form(True)` **Returns:** HTML addrecipients() · L2273

POST `/submit-alert-rule`

Submit Alert Rule

Args: `issue_id: str = Form(PydanticUndefined); rootcause_number: int = Form(1); event_rule_id: int = Form(PydanticUndefined); row_id: int = Form(None)` **Returns:** HTML submit_alert_rule() · L4320

POST `/submit-alert-threshold`

Submit Alert Threshold

Args: `alert_id: int = Form(PydanticUndefined); level: int = Form(3); value_start: float = Form(0); value_end: float = Form(100); row_id: int = Form(None)` **Returns:** HTML submit_alert_threshold() · L4233

POST `/submit-generate_report`

Report Capture

Args: `report: str = Form(PydanticUndefined); recipients: str = Form(PydanticUndefined); referer: str = Form(PydanticUndefined)` **Returns:** HTML report_capture() · L3551

POST /submit-global-param

Submit Global Param

Args: `key: str = Form(PydanticUndefined); value: str = Form()` **Returns:** HTML submit_global_param() · L4379

POST /submit-mailconfiguration

Mailconfigure

Args: `smtp_server: str = Form(PydanticUndefined); smtp_user: str = Form(PydanticUndefined); smtp_password: str = Form(PydanticUndefined); smtp_port: int = Form(PydanticUndefined); tls: bool = Form(False); smtp_sender: str = Form(PydanticUndefined); referer: str = Form(); row_id: str = Form(None); group_name: str = Form(); recipients: str = Form(); group_id: str = Form(None)` **Returns:** HTML mailconfigure() · L541

POST /submit-processform

Show Processform

Args: `process_name: str = Form(PydanticUndefined); interval: int = Form(PydanticUndefined); active: bool = Form(PydanticUndefined)` **Returns:** HTML show_processform() · L3136

POST /submit-report-schedule

Create or update a report schedule.

Args: `report_id: str = Form(None); report_name: str = Form(None); schedule: str = Form(daily); occurs_at: str = Form(06:00); recipients: str = Form(None); report_type: str = Form(json); report_query: str = Form(None); report_url: str = Form(None); is_active: str = Form(true); server_name: str = Form(None)` **Returns:** HTML submit_report_schedule() · L3905

POST /submit-report_capture

Report Capture

Args: `report: str = Form(PydanticUndefined); url: str = Form(PydanticUndefined); recipients: str = Form(PydanticUndefined); referer: str = Form(PydanticUndefined)` **Returns:** HTML report_capture() · L3621

POST /submit-reportbymail

Submit Mail Report

Args: `server: str = Form(PydanticUndefined); start_time: str = Form(PydanticUndefined); end_time: str = Form(PydanticUndefined); recipients: str = Form(PydanticUndefined); reportname: str = Form(PydanticUndefined); referer: str = Form(PydanticUndefined)` **Returns:** HTML submit_mail_report() · L2744

POST /submit-risklevel

Submit Risklevel

Args: `none` **Returns:** HTML submit_risklevel() · L2624

POST /submit-serverform

Show Server Form

Args: `server: str = Form(PydanticUndefined); add_delete_modify: str = Form(PydanticUndefined); ipAddress: str = Form(PydanticUndefined); port: int = Form(PydanticUndefined); dbVendor: str = Form(PydanticUndefined); dbVersion: str = Form(); authType: str = Form(PydanticUndefined); user: str = Form(); password: str = Form(); service_name: str = Form()` **Returns:** HTML show_server_form() · L3370

POST /submit-webhook-alert

Submit Webhook Alert

Args: `metric_type: str = Form(PydanticUndefined); send_mail_alert: str = Form(false); send_siem_alert: str = Form(false); send_diagnosis_evidence: str = Form(false); row_id: int = Form(None)` **Returns:** HTML submit_webhook_alert() · L4435

POST /submit_custom_metric_enable

Submit Custom Metric Enable

Args: `row_id: str = Form(PydanticUndefined)` **Returns:** HTML submit_custom_metric_enable() · L2891

POST /submit_custom_metrics

Submit Custom Metrics

Args: `metric_name: str = Form(PydanticUndefined); metric_desc: str = Form(PydanticUndefined); query: str = Form(PydanticUndefined); db_vendor_selector: str = Form(PydanticUndefined); action_select: int = Form(PydanticUndefined)` **Returns:** HTML submit_custom_metrics() · L2844

POST /submit_custom_metrics_update

Show Custom Metrics Update

Args: `row_id: int` **Returns:** HTML show_custom_metrics_update() · L3176

POST /submit_email_configuration

Submit Email Configuration

Args: `smtp_server: str = Form(PydanticUndefined); smtp_port: int = Form(PydanticUndefined); smtp_user: str = Form(PydanticUndefined); smtp_password: str = Form(PydanticUndefined); tls: bool = Form(PydanticUndefined)` **Returns:** HTML submit_email_configuration() · L3057

POST /submit_metrics_enable

Submit Metrics Enable

Args: `action_select: str = Form(PydanticUndefined); row_id: int = Form(PydanticUndefined)` **Returns:** HTML
submit_metrics_enable() · L2939

POST `/submit_metrics_enable_all_servers`

Submit Metrics Enable

Args: `action_select: str = Form(PydanticUndefined); row_id: int = Form(PydanticUndefined)` **Returns:** HTML
submit_metrics_enable() · L2980

POST `/submit_retention_policy`

Submit

Args: `row_id: int` **Returns:** HTML submit() · L3110

POST `/submit_siem_configuration`

Show Siem Configuration

Args: `select_vendor: str = Form(PydanticUndefined); url: str = Form(PydanticUndefined); secret: str = Form(); cs_mode: str = Form(logscale); cs_port: str = Form(); wz_proto: str = Form(syslog); wz_port: str = Form(514); gen_format: str = Form(); gen_transport: str = Form(); gen_port: str = Form(514)` **Returns:** HTML show_siem_configuration() · L3446

POST `/test-connection`

Test Connection

Args: `ipAddress: str = Form(PydanticUndefined); port: str = Form(); dbVendor: str = Form(PydanticUndefined); authType: str = Form(sql); user: str = Form(); password: str = Form(); service_name: str = Form()` **Returns:** HTML test_connection() · L3310